



Lecture 12

Compiler Design CSC-305

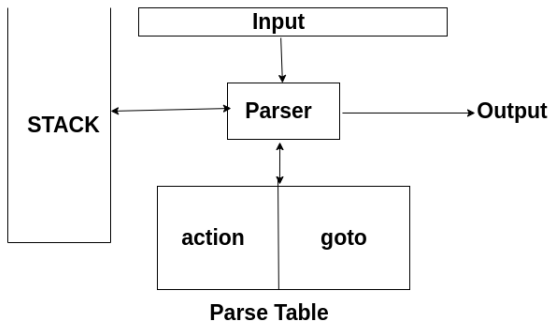
Awanish Pandey

Department of Computer Science and Engineering
Indian Institute of Technology
Roorkee

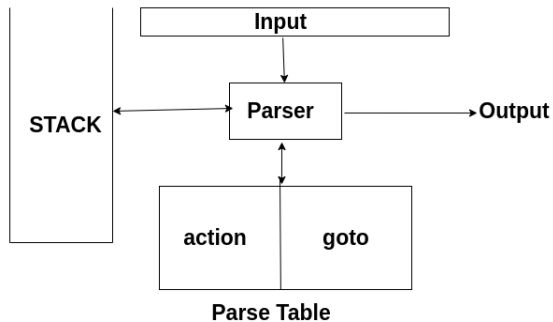
August 17, 2026

LR Parsing

LR Parsing

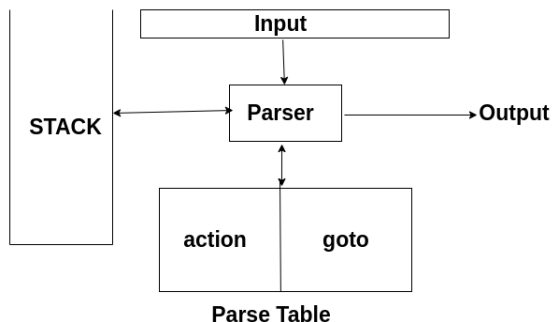


LR Parsing



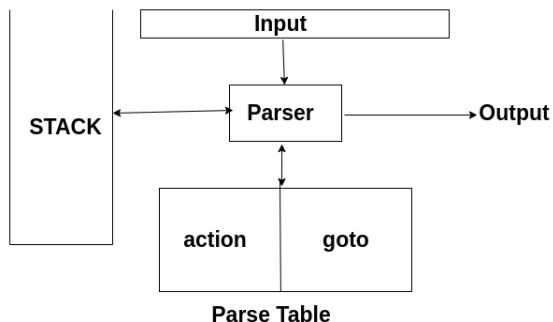
- Input contains the input string.

LR Parsing



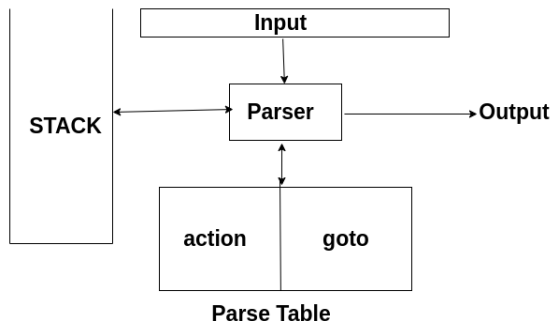
- Input contains the input string.
- Stack contains a string of the form $S_0X_1S_1X_2\cdots X_nS_n$ where each X_i is a grammar symbol and each S_i is a state.

LR Parsing



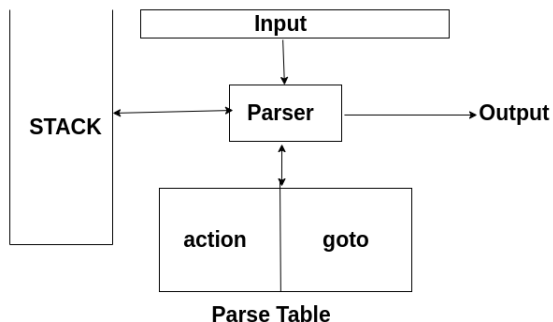
- Input contains the input string.
- Stack contains a string of the form $S_0X_1S_1X_2\cdots X_nS_n$ where each X_i is a grammar symbol and each S_i is a state.
- Tables contain **action** and **goto** parts.

LR Parsing



- Input contains the input string.
- Stack contains a string of the form $S_0X_1S_1X_2\cdots X_nS_n$ where each X_i is a grammar symbol and each S_i is a state.
- Tables contain **action** and **goto** parts.
- action table is indexed by state and terminal symbols.

LR Parsing



- Input contains the input string.
- Stack contains a string of the form $S_0X_1S_1X_2\cdots X_nS_n$ where each X_i is a grammar symbol and each S_i is a state.
- Tables contain **action** and **goto** parts.
- action table is indexed by state and terminal symbols.
- goto table is indexed by state and non terminal symbols.

Actions in an LR (shift reduce) parser

Actions in an LR (shift reduce) parser

- Assume S_i is top of stack and a_j is current input symbol

Actions in an LR (shift reduce) parser

- Assume S_i is top of stack and a_j is current input symbol
- Action $[S_i, a_j]$ can have four values

Actions in an LR (shift reduce) parser

- Assume S_i is top of stack and a_j is current input symbol
- Action $[S_i, a_j]$ can have four values
 - 1 shift a_j to the stack and goto state S_k

Actions in an LR (shift reduce) parser

- Assume S_i is top of stack and a_j is current input symbol
- Action $[S_i, a_j]$ can have four values
 - ① shift a_j to the stack and goto state S_k
 - ② reduce by a rule

Actions in an LR (shift reduce) parser

- Assume S_i is top of stack and a_j is current input symbol
- Action $[S_i, a_j]$ can have four values
 - ① shift a_j to the stack and goto state S_k
 - ② reduce by a rule
 - ③ accept

Actions in an LR (shift reduce) parser

- Assume S_i is top of stack and a_j is current input symbol
- Action $[S_i, a_j]$ can have four values
 - ① shift a_j to the stack and goto state S_k
 - ② reduce by a rule
 - ③ accept
 - ④ error

Configurations in LR parser

Stack : $S_0X_1S_1X_2\cdots X_mS_m$ *Input* : $a_ia_{i+1}\cdots a_n\$$

Configurations in LR parser

Stack : $S_0X_1S_1X_2\cdots X_mS_m$ *Input* : $a_ia_{i+1}\cdots a_n\$$

- If $action[S_m, a_i] = shift\ S$

Configurations in LR parser

Stack : $S_0X_1S_1X_2\cdots X_mS_m$ Input : $a_ia_{i+1}\cdots a_n\$$

- If $action[S_m, a_i] = shift\ S$

Then the configuration becomes

Stack : $S_0X_1S_1\cdots X_mS_ma_iS$ Input : $a_{i+1}\cdots a_n\$$

Configurations in LR parser

Stack : $S_0X_1S_1X_2\cdots X_mS_m$ *Input* : $a_ia_{i+1}\cdots a_n\$$

- If $action[S_m, a_i] = \text{shift } S$

Then the configuration becomes

Stack : $S_0X_1S_1\cdots X_mS_ma_iS$ *Input* : $a_{i+1}\cdots a_n\$$

- If $action[S_m, a_i] = \text{reduce } A \rightarrow \beta$

Configurations in LR parser

Stack : $S_0X_1S_1X_2\cdots X_mS_m$ *Input* : $a_ia_{i+1}\cdots a_n\$$

- If $action[S_m, a_i] = \text{shift } S$

Then the configuration becomes

Stack : $S_0X_1S_1\cdots X_mS_ma_iS$ *Input* : $a_{i+1}\cdots a_n\$$

- If $action[S_m, a_i] = \text{reduce } A \rightarrow \beta$

Then the configuration becomes

Stack : $S_0X_1S_1\cdots S_{m-r}X_{m-r+1}S$ *Input* : $a_ia_{i+1}\cdots a_n\$$

Configurations in LR parser

Stack : $S_0X_1S_1X_2\cdots X_mS_m$ *Input* : $a_ia_{i+1}\cdots a_n\$$

- If $action[S_m, a_i] = \text{shift } S$

Then the configuration becomes

Stack : $S_0X_1S_1\cdots X_mS_ma_iS$ *Input* : $a_{i+1}\cdots a_n\$$

- If $action[S_m, a_i] = \text{reduce } A \rightarrow \beta$

Then the configuration becomes

Stack : $S_0X_1S_1\cdots S_{m-r}X_{m-r+1}S$ *Input* : $a_ia_{i+1}\cdots a_n\$$

Where $r = |\beta|$ and $S = goto[S_{m-r}, A]$

Configurations in LR parser

Stack : $S_0X_1S_1X_2\cdots X_mS_m$ *Input* : $a_ia_{i+1}\cdots a_n\$$

- If $action[S_m, a_i] = \text{shift } S$

Then the configuration becomes

Stack : $S_0X_1S_1\cdots X_mS_ma_iS$ *Input* : $a_{i+1}\cdots a_n\$$

- If $action[S_m, a_i] = \text{reduce } A \rightarrow \beta$

Then the configuration becomes

Stack : $S_0X_1S_1\cdots S_{m-r}X_{m-r+1}S$ *Input* : $a_ia_{i+1}\cdots a_n\$$

Where $r = |\beta|$ and $S = \text{goto}[S_{m-r}, A]$

- If $action[S_m, a_i] = \text{accept}$

Configurations in LR parser

Stack : $S_0X_1S_1X_2\cdots X_mS_m$ *Input* : $a_ia_{i+1}\cdots a_n\$$

- If $action[S_m, a_i] = \text{shift } S$

Then the configuration becomes

Stack : $S_0X_1S_1\cdots X_mS_ma_iS$ *Input* : $a_{i+1}\cdots a_n\$$

- If $action[S_m, a_i] = \text{reduce } A \rightarrow \beta$

Then the configuration becomes

Stack : $S_0X_1S_1\cdots S_{m-r}X_{m-r+1}S$ *Input* : $a_ia_{i+1}\cdots a_n\$$

Where $r = |\beta|$ and $S = \text{goto}[S_{m-r}, A]$

- If $action[S_m, a_i] = \text{accept}$

Then parsing is completed. *HALT*

Configurations in LR parser

Stack : $S_0X_1S_1X_2\cdots X_mS_m$ *Input* : $a_ia_{i+1}\cdots a_n\$$

- If $action[S_m, a_i] = \text{shift } S$

Then the configuration becomes

Stack : $S_0X_1S_1\cdots X_mS_ma_iS$ *Input* : $a_{i+1}\cdots a_n\$$

- If $action[S_m, a_i] = \text{reduce } A \rightarrow \beta$

Then the configuration becomes

Stack : $S_0X_1S_1\cdots S_{m-r}X_{m-r+1}S$ *Input* : $a_ia_{i+1}\cdots a_n\$$

Where $r = |\beta|$ and $S = \text{goto}[S_{m-r}, A]$

- If $action[S_m, a_i] = \text{accept}$

Then parsing is completed. *HALT*

- If $action[S_m, a_i] = \text{error}$

Configurations in LR parser

Stack : $S_0X_1S_1X_2\cdots X_mS_m$ *Input* : $a_ia_{i+1}\cdots a_n\$$

- If $action[S_m, a_i] = \text{shift } S$

Then the configuration becomes

Stack : $S_0X_1S_1\cdots X_mS_ma_iS$ *Input* : $a_{i+1}\cdots a_n\$$

- If $action[S_m, a_i] = \text{reduce } A \rightarrow \beta$

Then the configuration becomes

Stack : $S_0X_1S_1\cdots S_{m-r}X_{m-r+1}S$ *Input* : $a_ia_{i+1}\cdots a_n\$$

Where $r = |\beta|$ and $S = \text{goto}[S_{m-r}, A]$

- If $action[S_m, a_i] = \text{accept}$

Then parsing is completed. *HALT*

- If $action[S_m, a_i] = \text{error}$

Then invoke error recovery routine

LR parsing Algorithm

Algorithm LR Parsing Algorithm

```
1: Initial State Stack :  $S_0$    Input :  $w\$$ 
2: for TRUE do
3:   if  $\text{action}[S, a] = \text{shift } S'$  then
4:      $\text{push}(a); \text{push}(S'); \text{ip}++$ 
5:   else if  $\text{action}[S, a] = \text{reduce } A \rightarrow \beta$  then
6:      $\text{pop}(2 * |\beta|)$  symbols;
7:      $\text{push}(A); \text{push}(\text{goto}[S'', A])$  { $S''$  is the state after popping symbols}
8:   else if  $\text{action}[S, a] = \text{accept}$  then
9:      $\text{exit}$ 
10:  else
11:     $\text{error}()$ 
12:  end if
13: end for
```

Example

- Consider the grammar

Example

- Consider the grammar $E \rightarrow E + T \mid T$ $T \rightarrow T * F \mid F$ $F \rightarrow (E) \mid id$

state	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Parse $\text{id} + \text{id} * \text{id}$

stack	input	action
0	$\text{id} + \text{id} * \text{id}\$$	s5
0id5	$+ \text{id} * \text{id}\$$	reduce $F \rightarrow \text{id}$
0F3	$+ \text{id} * \text{id}\$$	reduce $T \rightarrow F$
0T2	$+ \text{id} * \text{id}\$$	reduce $E \rightarrow T$
0E1	$+ \text{id} * \text{id}\$$	shift 6
0E1+6	$\text{id} * \text{id} \$$	shift 5
0E1+6id5	$* \text{id} \$$	reduce $F \rightarrow \text{id}$
0E1+6F3	$* \text{id} \$$	reduce $T \rightarrow F$
0E1+6T9	$* \text{id} \$$	shift 7
0E1+6T9*7	$\text{id} \$$	shift 5
0E1+6T9*7id5	$\$$	reduce $F \rightarrow \text{id}$
0E1+6T9*7F10	$\$$	reduce $T \rightarrow T * F$
0E1+6T9	$\$$	reduce $E \rightarrow E + T$
0E1	$\$$	acc

Parser states

Parser states

- Goal is to know the valid reductions at any given point

Parser states

- Goal is to know the valid reductions at any given point
- Summarize all possible stack prefixes α as a parser state

Parser states

- Goal is to know the valid reductions at any given point
- Summarize all possible stack prefixes α as a parser state
- Parser state is defined by a DFA state that reads in the stack α

Parser states

- Goal is to know the valid reductions at any given point
- Summarize all possible stack prefixes α as a parser state
- Parser state is defined by a DFA state that reads in the stack α
- Accept states of DFA are unique reductions

Constructing parse table

Constructing parse table

- Augment the grammar

Constructing parse table

- Augment the grammar
 - ▶ G is a grammar with start symbol S

Constructing parse table

- Augment the grammar
 - ▶ G is a grammar with start symbol S
 - ▶ The augmented grammar G' for G has a new start symbol S' and an additional production $S' \rightarrow S$

Constructing parse table

- Augment the grammar
 - ▶ G is a grammar with start symbol S
 - ▶ The augmented grammar G' for G has a new start symbol S' and an additional production $S' \rightarrow S$
 - ▶ When the parser reduces by this rule it will stop with accept

Constructing parse table

- Augment the grammar
 - ▶ G is a grammar with start symbol S
 - ▶ The augmented grammar G' for G has a new start symbol S' and an additional production $S' \rightarrow S$
 - ▶ When the parser reduces by this rule it will stop with accept
- Viable prefixes

Constructing parse table

- Augment the grammar
 - ▶ G is a grammar with start symbol S
 - ▶ The augmented grammar G' for G has a new start symbol S' and an additional production $S' \rightarrow S$
 - ▶ When the parser reduces by this rule it will stop with accept
- Viable prefixes
 - ▶ α is a viable prefix of the grammar if

Constructing parse table

- Augment the grammar
 - ▶ G is a grammar with start symbol S
 - ▶ The augmented grammar G' for G has a new start symbol S' and an additional production $S' \rightarrow S$
 - ▶ When the parser reduces by this rule it will stop with accept
- Viable prefixes
 - ▶ α is a viable prefix of the grammar if
 - ★ There is a w such that αw is a right sentential form.

Constructing parse table

- Augment the grammar
 - ▶ G is a grammar with start symbol S
 - ▶ The augmented grammar G' for G has a new start symbol S' and an additional production $S' \rightarrow S$
 - ▶ When the parser reduces by this rule it will stop with accept
- Viable prefixes
 - ▶ α is a viable prefix of the grammar if
 - ★ There is a w such that αw is a right sentential form.
 - ★ $\alpha.w$ is a configuration of the shift reduce parser

Constructing parse table

- Augment the grammar
 - ▶ G is a grammar with start symbol S
 - ▶ The augmented grammar G' for G has a new start symbol S' and an additional production $S' \rightarrow S$
 - ▶ When the parser reduces by this rule it will stop with accept
- Viable prefixes
 - ▶ α is a viable prefix of the grammar if
 - ★ There is a w such that αw is a right sentential form.
 - ★ $\alpha.w$ is a configuration of the shift reduce parser
 - ▶ As long as the parser has viable prefixes on the stack no parser error has been seen

Constructing parse table

- Augment the grammar
 - ▶ G is a grammar with start symbol S
 - ▶ The augmented grammar G' for G has a new start symbol S' and an additional production $S' \rightarrow S$
 - ▶ When the parser reduces by this rule it will stop with accept
- Viable prefixes
 - ▶ α is a viable prefix of the grammar if
 - ★ There is a w such that αw is a right sentential form.
 - ★ $\alpha.w$ is a configuration of the shift reduce parser
 - ▶ As long as the parser has viable prefixes on the stack no parser error has been seen
 - ▶ The set of viable prefixes is a regular language

Constructing parse table

- Augment the grammar
 - ▶ G is a grammar with start symbol S
 - ▶ The augmented grammar G' for G has a new start symbol S' and an additional production $S' \rightarrow S$
 - ▶ When the parser reduces by this rule it will stop with accept
- Viable prefixes
 - ▶ α is a viable prefix of the grammar if
 - ★ There is a w such that αw is a right sentential form.
 - ★ $\alpha.w$ is a configuration of the shift reduce parser
 - ▶ As long as the parser has viable prefixes on the stack no parser error has been seen
 - ▶ The set of viable prefixes is a regular language
 - ▶ Construct an automaton that accepts viable prefixes

LR(0) items

LR(0) items

- An LR(0) item of a grammar G is a production of G with a special symbol $\cdot$ at some position of the right side

LR(0) items

- An LR(0) item of a grammar G is a production of G with a special symbol "." at some position of the right side
- Production $A \rightarrow XYZ$ gives four LR(0) items

LR(0) items

- An LR(0) item of a grammar G is a production of G with a special symbol "." at some position of the right side
- Production $A \rightarrow XYZ$ gives four LR(0) items
 $A \rightarrow .XYZ$

LR(0) items

- An LR(0) item of a grammar G is a production of G with a special symbol "." at some position of the right side
- Production $A \rightarrow XYZ$ gives four LR(0) items
$$A \rightarrow .XYZ$$
$$A \rightarrow X.YZ$$

LR(0) items

- An LR(0) item of a grammar G is a production of G with a special symbol "." at some position of the right side
- Production $A \rightarrow XYZ$ gives four LR(0) items
$$A \rightarrow .XYZ$$
$$A \rightarrow X.YZ$$
$$A \rightarrow XY.Z$$

LR(0) items

- An LR(0) item of a grammar G is a production of G with a special symbol "." at some position of the right side
- Production $A \rightarrow XYZ$ gives four LR(0) items
$$A \rightarrow .XYZ$$
$$A \rightarrow X.YZ$$
$$A \rightarrow XY.Z$$
$$A \rightarrow XYZ.$$

LR(0) items

- An LR(0) item of a grammar G is a production of G with a special symbol "." at some position of the right side
- Production $A \rightarrow XYZ$ gives four LR(0) items
$$A \rightarrow .XYZ$$
$$A \rightarrow X.YZ$$
$$A \rightarrow XY.Z$$
$$A \rightarrow XYZ.$$
- An item indicates how much of a production has been seen at a point in the process of parsing

LR(0) items

- An LR(0) item of a grammar G is a production of G with a special symbol "." at some position of the right side
- Production $A \rightarrow XYZ$ gives four LR(0) items
$$A \rightarrow .XYZ$$
$$A \rightarrow X.YZ$$
$$A \rightarrow XY.Z$$
$$A \rightarrow XYZ.$$
- An item indicates how much of a production has been seen at a point in the process of parsing
 - ▶ Symbols on the left of "." are already on the stacks

LR(0) items

- An LR(0) item of a grammar G is a production of G with a special symbol "." at some position of the right side
- Production $A \rightarrow XYZ$ gives four LR(0) items
$$A \rightarrow .XYZ$$
$$A \rightarrow X.YZ$$
$$A \rightarrow XY.Z$$
$$A \rightarrow XYZ.$$
- An item indicates how much of a production has been seen at a point in the process of parsing
 - ▶ Symbols on the left of "." are already on the stacks
 - ▶ Symbols on the right of "." are expected in the input

Start State

Start State

- Start state of DFA is an empty stack corresponding to $S' \rightarrow .S$ item

Start State

- Start state of DFA is an empty stack corresponding to $S' \rightarrow .S$ item
 - ▶ This means no input has been seen

Start State

- Start state of DFA is an empty stack corresponding to $S' \rightarrow .S$ item
 - ▶ This means no input has been seen
 - ▶ The parser expects to see a string derived from S

Start State

- Start state of DFA is an empty stack corresponding to $S' \rightarrow .S$ item
 - ▶ This means no input has been seen
 - ▶ The parser expects to see a string derived from S
- Closure of a state adds items for all productions whose LHS occurs in an item in the state, just after "."

Start State

- Start state of DFA is an empty stack corresponding to $S' \rightarrow .S$ item
 - ▶ This means no input has been seen
 - ▶ The parser expects to see a string derived from S
- Closure of a state adds items for all productions whose LHS occurs in an item in the state, just after "."
 - ▶ Set of possible productions to be reduced next

Start State

- Start state of DFA is an empty stack corresponding to $S' \rightarrow .S$ item
 - ▶ This means no input has been seen
 - ▶ The parser expects to see a string derived from S
- Closure of a state adds items for all productions whose LHS occurs in an item in the state, just after "."
 - ▶ Set of possible productions to be reduced next
 - ▶ Added items have "." located at the beginning

Start State

- Start state of DFA is an empty stack corresponding to $S' \rightarrow .S$ item
 - ▶ This means no input has been seen
 - ▶ The parser expects to see a string derived from S
- Closure of a state adds items for all productions whose LHS occurs in an item in the state, just after "."
 - ▶ Set of possible productions to be reduced next
 - ▶ Added items have "." located at the beginning
 - ▶ No symbol of these items is on the stack as yet

Example

- Consider the grammar

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

Example

- Consider the grammar

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

- If I is $E' \rightarrow .E$ then $\text{closure}(I)$ is

Example

- Consider the grammar

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

- If I is $E' \rightarrow .E$ then $\text{closure}(I)$ is
 $E' \rightarrow .E$

Example

- Consider the grammar

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

- If I is $E' \rightarrow .E$ then $\text{closure}(I)$ is

$$E' \rightarrow .E$$

$$E \rightarrow .E + T$$

Example

- Consider the grammar

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

- If I is $E' \rightarrow .E$ then $\text{closure}(I)$ is

$$E' \rightarrow .E$$

$$E \rightarrow .E + T$$

$$E \rightarrow .T$$

Example

- Consider the grammar

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

- If I is $E' \rightarrow .E$ then $\text{closure}(I)$ is

$$E' \rightarrow .E$$

$$E \rightarrow .E + T$$

$$E \rightarrow .T$$

$$T \rightarrow .T * F$$

Example

- Consider the grammar

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

- If I is $E' \rightarrow .E$ then $\text{closure}(I)$ is

$$E' \rightarrow .E$$

$$E \rightarrow .E + T$$

$$E \rightarrow .T$$

$$T \rightarrow .T * F$$

$$T \rightarrow .F$$

Example

- Consider the grammar

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

- If I is $E' \rightarrow .E$ then $\text{closure}(I)$ is

$$E' \rightarrow .E$$

$$E \rightarrow .E + T$$

$$E \rightarrow .T$$

$$T \rightarrow .T * F$$

$$T \rightarrow .F$$

$$F \rightarrow .id$$

Example

- Consider the grammar

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

- If I is $E' \rightarrow .E$ then $\text{closure}(I)$ is

$$E' \rightarrow .E$$

$$E \rightarrow .E + T$$

$$E \rightarrow .T$$

$$T \rightarrow .T * F$$

$$T \rightarrow .F$$

$$F \rightarrow .id$$

$$F \rightarrow .(E)$$

Example

- Grammar:

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

Example

- Grammar:

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

- $I_0 : \text{closure}(E' \rightarrow .E)$

Example

- Grammar:

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

- $I_0 : \text{closure}(E' \rightarrow .E)$

$$E' \rightarrow .E$$

$$E \rightarrow .E + T$$

$$E \rightarrow .T$$

$$T \rightarrow .T * F$$

$$T \rightarrow .F$$

$$F \rightarrow .(E)$$

$$F \rightarrow .id$$

Example

- Grammar:

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

- $l_0 : \text{closure}(E' \rightarrow .E)$

$$E' \rightarrow .E$$

$$E \rightarrow .E + T$$

$$E \rightarrow .T$$

$$T \rightarrow .T * F$$

$$T \rightarrow .F$$

$$F \rightarrow .(E)$$

$$F \rightarrow .id$$

- $l_1 : \text{goto}(l_0, E)$

Example

- Grammar:

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

- $l_0 : \text{closure}(E' \rightarrow .E)$

$$E' \rightarrow .E$$

$$E \rightarrow .E + T$$

$$E \rightarrow .T$$

$$T \rightarrow .T * F$$

$$T \rightarrow .F$$

$$F \rightarrow .(E)$$

$$F \rightarrow .id$$

- $l_1 : \text{goto}(l_0, E)$

$$E' \rightarrow E.$$

$$E \rightarrow E. + T$$

Example

- Grammar:

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

- $l_0 : \text{closure}(E' \rightarrow .E)$

$$E' \rightarrow .E$$

$$E \rightarrow .E + T$$

$$E \rightarrow .T$$

$$T \rightarrow .T * F$$

$$T \rightarrow .F$$

$$F \rightarrow .(E)$$

$$F \rightarrow .id$$

- $l_1 : \text{goto}(l_0, E)$

$$E' \rightarrow E.$$

$$E \rightarrow E. + T$$

- $l_2 : \text{goto}(l_0, T)$

Example

- Grammar:

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

- $l_0 : \text{closure}(E' \rightarrow .E)$

$$E' \rightarrow .E$$

$$E \rightarrow .E + T$$

$$E \rightarrow .T$$

$$T \rightarrow .T * F$$

$$T \rightarrow .F$$

$$F \rightarrow .(E)$$

$$F \rightarrow .id$$

- $l_1 : \text{goto}(l_0, E)$

$$E' \rightarrow E.$$

$$E \rightarrow E. + T$$

- $l_2 : \text{goto}(l_0, T)$

$$E \rightarrow T.$$

$$T \rightarrow T. * F$$

Example

- Grammar:

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

- $l_0 : \text{closure}(E' \rightarrow .E)$

$$E' \rightarrow .E$$

$$E \rightarrow .E + T$$

$$E \rightarrow .T$$

$$T \rightarrow .T * F$$

$$T \rightarrow .F$$

$$F \rightarrow .(E)$$

$$F \rightarrow .id$$

- $l_1 : \text{goto}(l_0, E)$

$$E' \rightarrow E.$$

$$E \rightarrow E. + T$$

- $l_2 : \text{goto}(l_0, T)$

$$E \rightarrow T.$$

$$T \rightarrow T. * F$$

- $l_3 : \text{goto}(l_0, F)$

Example

- Grammar:

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

- $l_0 : \text{closure}(E' \rightarrow .E)$

$$E' \rightarrow .E$$

$$E \rightarrow .E + T$$

$$E \rightarrow .T$$

$$T \rightarrow .T * F$$

$$T \rightarrow .F$$

$$F \rightarrow .(E)$$

$$F \rightarrow .id$$

- $l_1 : \text{goto}(l_0, E)$

$$E' \rightarrow E.$$

$$E \rightarrow E. + T$$

- $l_2 : \text{goto}(l_0, T)$

$$E \rightarrow T.$$

$$T \rightarrow T. * F$$

- $l_3 : \text{goto}(l_0, F)$

$$T \rightarrow F.$$

Example

- Grammar:

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

- $l_0 : \text{closure}(E' \rightarrow .E)$

$$E' \rightarrow .E$$

$$E \rightarrow .E + T$$

$$E \rightarrow .T$$

$$T \rightarrow .T * F$$

$$T \rightarrow .F$$

$$F \rightarrow .(E)$$

$$F \rightarrow .id$$

- $l_1 : \text{goto}(l_0, E)$

$$E' \rightarrow E.$$

$$E \rightarrow E. + T$$

- $l_2 : \text{goto}(l_0, T)$

$$E \rightarrow T.$$

$$T \rightarrow T. * F$$

- $l_3 : \text{goto}(l_0, F)$

$$T \rightarrow F.$$

- $l_4 : \text{goto}(l_0, ($

Example

- Grammar:

$E' \rightarrow E$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

- $l_0 : \text{closure}(E' \rightarrow .E)$

$E' \rightarrow .E$

$E \rightarrow .E + T$

$E \rightarrow .T$

$T \rightarrow .T * F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

- $l_1 : \text{goto}(l_0, E)$

$E' \rightarrow E.$

$E \rightarrow E. + T$

- $l_2 : \text{goto}(l_0, T)$

$E \rightarrow T.$

$T \rightarrow T. * F$

- $l_3 : \text{goto}(l_0, F)$

$T \rightarrow F.$

- $l_4 : \text{goto}(l_0, ($

$F \rightarrow (.E)$

$E \rightarrow .E + T$

$E \rightarrow .T$

$T \rightarrow .T * F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

Example

- Grammar:

$E' \rightarrow E$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

- $l_0 : \text{closure}(E' \rightarrow .E)$

$E' \rightarrow .E$

$E \rightarrow .E + T$

$E \rightarrow .T$

$T \rightarrow .T * F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

- $l_1 : \text{goto}(l_0, E)$

$E' \rightarrow E.$

$E \rightarrow E. + T$

- $l_2 : \text{goto}(l_0, T)$

$E \rightarrow T.$

$T \rightarrow T. * F$

- $l_3 : \text{goto}(l_0, F)$

$T \rightarrow F.$

- $l_4 : \text{goto}(l_0, ($

$F \rightarrow (.E)$

$E \rightarrow .E + T$

$E \rightarrow .T$

$T \rightarrow .T * F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

- $l_5 : \text{goto}(l_0, id)$

Example

- Grammar:

$E' \rightarrow E$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

- $l_0 : \text{closure}(E' \rightarrow .E)$

$E' \rightarrow .E$

$E \rightarrow .E + T$

$E \rightarrow .T$

$T \rightarrow .T * F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

- $l_1 : \text{goto}(l_0, E)$

$E' \rightarrow E.$

$E \rightarrow E. + T$

- $l_2 : \text{goto}(l_0, T)$

$E \rightarrow T.$

$T \rightarrow T. * F$

- $l_3 : \text{goto}(l_0, F)$

$T \rightarrow F.$

- $l_4 : \text{goto}(l_0, ($

$F \rightarrow (.E)$

$E \rightarrow .E + T$

$E \rightarrow .T$

$T \rightarrow .T * F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

- $l_5 : \text{goto}(l_0, id)$

$F \rightarrow id.$

Example

- Grammar:

$E' \rightarrow E$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

- $l_0 : \text{closure}(E' \rightarrow .E)$

$E' \rightarrow .E$

$E \rightarrow .E + T$

$E \rightarrow .T$

$T \rightarrow .T * F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

- $l_1 : \text{goto}(l_0, E)$

$E' \rightarrow E.$

$E \rightarrow E. + T$

- $l_2 : \text{goto}(l_0, T)$

$E \rightarrow T.$

$T \rightarrow T. * F$

- $l_3 : \text{goto}(l_0, F)$

$T \rightarrow F.$

- $l_4 : \text{goto}(l_0, ($

$F \rightarrow (.E)$

$E \rightarrow .E + T$

$E \rightarrow .T$

$T \rightarrow .T * F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

- $l_5 : \text{goto}(l_0, id)$

$F \rightarrow id.$

Example

- $l_6 : goto(l_1, +)$

Example

- $l_6 : goto(l_1, +)$

$E \rightarrow E + .T$

$T \rightarrow .T * F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

Example

- $l_6 : goto(l_1, +)$
 $E \rightarrow E + .T$
 $T \rightarrow .T * F$
 $T \rightarrow .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_7 : goto(l_2, *)$

Example

- $l_6 : goto(l_1, +)$

$E \rightarrow E + .T$

$T \rightarrow .T * F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

- $l_7 : goto(l_2, *)$

$T \rightarrow T * .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

Example

- $l_6 : goto(l_1, +)$
 $E \rightarrow E + .T$
 $T \rightarrow .T * F$
 $T \rightarrow .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_7 : goto(l_2, *)$
 $T \rightarrow T * .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_8 : goto(l_4, E)$

Example

- $l_6 : \text{goto}(l_1, +)$

$E \rightarrow E + .T$

$T \rightarrow .T * F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

- $l_7 : \text{goto}(l_2, *)$

$T \rightarrow T * .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

- $l_8 : \text{goto}(l_4, E)$

$F \rightarrow (E.)$

$E \rightarrow E. + T$

Example

- $l_6 : \text{goto}(l_1, +)$
 $E \rightarrow E + .T$
 $T \rightarrow .T * F$
 $T \rightarrow .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_7 : \text{goto}(l_2, *)$
 $T \rightarrow T * .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_8 : \text{goto}(l_4, E)$
 $F \rightarrow (E.)$
 $E \rightarrow E. + T$
- $\text{goto}(l_4, T) \text{ is } l_2$

Example

- $l_6 : \text{goto}(l_1, +)$
 $E \rightarrow E + .T$
 $T \rightarrow .T * F$
 $T \rightarrow .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_7 : \text{goto}(l_2, *)$
 $T \rightarrow T * .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_8 : \text{goto}(l_4, E)$
 $F \rightarrow (E.)$
 $E \rightarrow E. + T$
- $\text{goto}(l_4, T) \text{ is } l_2$
- $\text{goto}(l_4, F) \text{ is } l_3$

Example

- $l_6 : \text{goto}(l_1, +)$

$E \rightarrow E + .T$

$T \rightarrow .T * F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

- $l_7 : \text{goto}(l_2, *)$

$T \rightarrow T * .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

- $l_8 : \text{goto}(l_4, E)$

$F \rightarrow (E.)$

$E \rightarrow E. + T$

- $\text{goto}(l_4, T) \text{ is } l_2$

- $\text{goto}(l_4, F) \text{ is } l_3$

- $\text{goto}(l_4, ()) \text{ is } l_4$

Example

- $l_6 : \text{goto}(l_1, +)$
 $E \rightarrow E + .T$
 $T \rightarrow .T * F$
 $T \rightarrow .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_7 : \text{goto}(l_2, *)$
 $T \rightarrow T * .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_8 : \text{goto}(l_4, E)$
 $F \rightarrow (E.)$
 $E \rightarrow E. + T$
- $\text{goto}(l_4, T) \text{ is } l_2$
- $\text{goto}(l_4, F) \text{ is } l_3$
- $\text{goto}(l_4, () \text{ is } l_4$
- $\text{goto}(l_4, id) \text{ is } l_5$

Example

- $l_6 : \text{goto}(l_1, +)$
 $E \rightarrow E + .T$
 $T \rightarrow .T * F$
 $T \rightarrow .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_7 : \text{goto}(l_2, *)$
 $T \rightarrow T * .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_8 : \text{goto}(l_4, E)$
 $F \rightarrow (E.)$
 $E \rightarrow E. + T$
- $\text{goto}(l_4, T) \text{ is } l_2$
- $\text{goto}(l_4, F) \text{ is } l_3$
- $\text{goto}(l_4, () \text{ is } l_4$
- $\text{goto}(l_4, id) \text{ is } l_5$
- $l_9 : \text{goto}(l_6, T)$

Example

- $l_6 : \text{goto}(l_1, +)$
 $E \rightarrow E + .T$
 $T \rightarrow .T * F$
 $T \rightarrow .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_7 : \text{goto}(l_2, *)$
 $T \rightarrow T * .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_8 : \text{goto}(l_4, E)$
 $F \rightarrow (E.)$
 $E \rightarrow E. + T$
- $\text{goto}(l_4, T) \text{ is } l_2$
- $\text{goto}(l_4, F) \text{ is } l_3$
- $\text{goto}(l_4, () \text{ is } l_4$
- $\text{goto}(l_4, id) \text{ is } l_5$
- $l_9 : \text{goto}(l_6, T)$
 $E \rightarrow E + T.$
 $T \rightarrow T. * F$

Example

- $l_6 : \text{goto}(l_1, +)$
 $E \rightarrow E + .T$
 $T \rightarrow .T * F$
 $T \rightarrow .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_7 : \text{goto}(l_2, *)$
 $T \rightarrow T * .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_8 : \text{goto}(l_4, E)$
 $F \rightarrow (E.)$
 $E \rightarrow E. + T$
- $\text{goto}(l_4, T) \text{ is } l_2$
- $\text{goto}(l_4, F) \text{ is } l_3$
- $\text{goto}(l_4, () \text{ is } l_4$
- $\text{goto}(l_4, id) \text{ is } l_5$
- $l_9 : \text{goto}(l_6, T)$
 $E \rightarrow E + T.$
 $T \rightarrow T. * F$
- $\text{goto}(l_6, F) \text{ is } l_3$

Example

- $l_6 : \text{goto}(l_1, +)$
 $E \rightarrow E + .T$
 $T \rightarrow .T * F$
 $T \rightarrow .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_7 : \text{goto}(l_2, *)$
 $T \rightarrow T * .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_8 : \text{goto}(l_4, E)$
 $F \rightarrow (E.)$
 $E \rightarrow E. + T$
- $\text{goto}(l_4, T) \text{ is } l_2$
- $\text{goto}(l_4, F) \text{ is } l_3$
- $\text{goto}(l_4, () \text{ is } l_4$
- $\text{goto}(l_4, id) \text{ is } l_5$
- $l_9 : \text{goto}(l_6, T)$
 $E \rightarrow E + T.$
 $T \rightarrow T. * F$
- $\text{goto}(l_6, F) \text{ is } l_3$
- $\text{goto}(l_6, () \text{ is } l_4$

Example

- $l_6 : \text{goto}(l_1, +)$
 $E \rightarrow E + .T$
 $T \rightarrow .T * F$
 $T \rightarrow .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_7 : \text{goto}(l_2, *)$
 $T \rightarrow T * .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_8 : \text{goto}(l_4, E)$
 $F \rightarrow (E.)$
 $E \rightarrow E. + T$
- $\text{goto}(l_4, T) \text{ is } l_2$
- $\text{goto}(l_4, F) \text{ is } l_3$
- $\text{goto}(l_4, () \text{ is } l_4$
- $\text{goto}(l_4, id) \text{ is } l_5$
- $l_9 : \text{goto}(l_6, T)$
 $E \rightarrow E + T.$
 $T \rightarrow T. * F$
- $\text{goto}(l_6, F) \text{ is } l_3$
- $\text{goto}(l_6, () \text{ is } l_4$
- $\text{goto}(l_6, id) \text{ is } l_5$

Example

- $l_6 : \text{goto}(l_1, +)$
 $E \rightarrow E + .T$
 $T \rightarrow .T * F$
 $T \rightarrow .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_7 : \text{goto}(l_2, *)$
 $T \rightarrow T * .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_8 : \text{goto}(l_4, E)$
 $F \rightarrow (E.)$
 $E \rightarrow E. + T$
- $\text{goto}(l_4, T) \text{ is } l_2$
- $\text{goto}(l_4, F) \text{ is } l_3$
- $\text{goto}(l_4, () \text{ is } l_4$
- $\text{goto}(l_4, id) \text{ is } l_5$
- $l_9 : \text{goto}(l_6, T)$
 $E \rightarrow E + T.$
 $T \rightarrow T. * F$
- $\text{goto}(l_6, F) \text{ is } l_3$
- $\text{goto}(l_6, () \text{ is } l_4$
- $\text{goto}(l_6, id) \text{ is } l_5$
- $l_{10} : \text{goto}(l_7, F)$
 $T \rightarrow T * F.$

Example

- $l_6 : \text{goto}(l_1, +)$
 $E \rightarrow E + .T$
 $T \rightarrow .T * F$
 $T \rightarrow .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_7 : \text{goto}(l_2, *)$
 $T \rightarrow T * .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_8 : \text{goto}(l_4, E)$
 $F \rightarrow (E.)$
 $E \rightarrow E. + T$

- $\text{goto}(l_4, T) \text{ is } l_2$
- $\text{goto}(l_4, F) \text{ is } l_3$
- $\text{goto}(l_4, ()) \text{ is } l_4$
- $\text{goto}(l_4, id) \text{ is } l_5$
- $l_9 : \text{goto}(l_6, T)$
 $E \rightarrow E + T.$
 $T \rightarrow T. * F$
- $\text{goto}(l_6, F) \text{ is } l_3$
- $\text{goto}(l_6, ()) \text{ is } l_4$
- $\text{goto}(l_6, id) \text{ is } l_5$

- $l_{10} : \text{goto}(l_7, F)$
 $T \rightarrow T * F.$
- $\text{goto}(l_7, ()) \text{ is } l_4$
 $\text{goto}(l_7, id) \text{ is } l_5$

Example

- $l_6 : \text{goto}(l_1, +)$
 $E \rightarrow E + .T$
 $T \rightarrow .T * F$
 $T \rightarrow .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_7 : \text{goto}(l_2, *)$
 $T \rightarrow T * .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_8 : \text{goto}(l_4, E)$
 $F \rightarrow (E.)$
 $E \rightarrow E. + T$

- $\text{goto}(l_4, T) \text{ is } l_2$
- $\text{goto}(l_4, F) \text{ is } l_3$
- $\text{goto}(l_4, ()) \text{ is } l_4$
- $\text{goto}(l_4, id) \text{ is } l_5$
- $l_9 : \text{goto}(l_6, T)$
 $E \rightarrow E + T.$
 $T \rightarrow T. * F$
- $\text{goto}(l_6, F) \text{ is } l_3$
- $\text{goto}(l_6, ()) \text{ is } l_4$
- $\text{goto}(l_6, id) \text{ is } l_5$

- $l_{10} : \text{goto}(l_7, F)$
 $T \rightarrow T * F.$
- $\text{goto}(l_7, ()) \text{ is } l_4$
 $\text{goto}(l_7, id) \text{ is } l_5$
- $l_{11} : \text{goto}(l_8,))$
 $F \rightarrow (E).$

Example

- $l_6 : \text{goto}(l_1, +)$
 $E \rightarrow E + .T$
 $T \rightarrow .T * F$
 $T \rightarrow .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_7 : \text{goto}(l_2, *)$
 $T \rightarrow T * .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_8 : \text{goto}(l_4, E)$
 $F \rightarrow (E.)$
 $E \rightarrow E. + T$
- $\text{goto}(l_4, T) \text{ is } l_2$
- $\text{goto}(l_4, F) \text{ is } l_3$
- $\text{goto}(l_4, () \text{ is } l_4$
- $\text{goto}(l_4, id) \text{ is } l_5$
- $l_9 : \text{goto}(l_6, T)$
 $E \rightarrow E + T.$
 $T \rightarrow T. * F$
- $\text{goto}(l_6, F) \text{ is } l_3$
- $\text{goto}(l_6, () \text{ is } l_4$
- $\text{goto}(l_6, id) \text{ is } l_5$
- $l_{10} : \text{goto}(l_7, F)$
 $T \rightarrow T * F.$
- $\text{goto}(l_7, () \text{ is } l_4$
 $\text{goto}(l_7, id) \text{ is } l_5$
- $l_{11} : \text{goto}(l_8,))$
 $F \rightarrow (E).$
- $\text{goto}(l_8, +) \text{ is } l_6$

Example

- $l_6 : \text{goto}(l_1, +)$
 $E \rightarrow E + .T$
 $T \rightarrow .T * F$
 $T \rightarrow .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_7 : \text{goto}(l_2, *)$
 $T \rightarrow T * .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$
- $l_8 : \text{goto}(l_4, E)$
 $F \rightarrow (E.)$
 $E \rightarrow E. + T$
- $\text{goto}(l_4, T) \text{ is } l_2$
- $\text{goto}(l_4, F) \text{ is } l_3$
- $\text{goto}(l_4, ()) \text{ is } l_4$
- $\text{goto}(l_4, id) \text{ is } l_5$
- $l_9 : \text{goto}(l_6, T)$
 $E \rightarrow E + T.$
 $T \rightarrow T. * F$
- $\text{goto}(l_6, F) \text{ is } l_3$
- $\text{goto}(l_6, ()) \text{ is } l_4$
- $\text{goto}(l_6, id) \text{ is } l_5$
- $l_{10} : \text{goto}(l_7, F)$
 $T \rightarrow T * F.$
- $\text{goto}(l_7, ()) \text{ is } l_4$
 $\text{goto}(l_7, id) \text{ is } l_5$
- $l_{11} : \text{goto}(l_8,))$
 $F \rightarrow (E).$
- $\text{goto}(l_8, +) \text{ is } l_6$
- $\text{goto}(l_9, *) \text{ is } l_7$

Closure operation

Closure operation

- If I is a set of items for a grammar G then $\text{closure}(I)$ is a set constructed as follows:

Closure operation

- If I is a set of items for a grammar G then $\text{closure}(I)$ is a set constructed as follows:
 - ▶ Every item in I is in $\text{closure}(I)$

Closure operation

- If I is a set of items for a grammar G then $\text{closure}(I)$ is a set constructed as follows:
 - ▶ Every item in I is in $\text{closure}(I)$
 - ▶ If $A \rightarrow \alpha.B\beta$ is in $\text{closure}(I)$ and $B \rightarrow \gamma$ is a production then $B \rightarrow .\gamma$ is in $\text{closure}(I)$

Closure operation

- If I is a set of items for a grammar G then $\text{closure}(I)$ is a set constructed as follows:
 - ▶ Every item in I is in $\text{closure}(I)$
 - ▶ If $A \rightarrow \alpha.B\beta$ is in $\text{closure}(I)$ and $B \rightarrow \gamma$ is a production then $B \rightarrow .\gamma$ is in $\text{closure}(I)$
- Intuitively $A \rightarrow \alpha.B\beta$ indicates that we might see a string derivable from $B\beta$ as input

Closure operation

- If I is a set of items for a grammar G then $\text{closure}(I)$ is a set constructed as follows:
 - ▶ Every item in I is in $\text{closure}(I)$
 - ▶ If $A \rightarrow \alpha.B\beta$ is in $\text{closure}(I)$ and $B \rightarrow \gamma$ is a production then $B \rightarrow .\gamma$ is in $\text{closure}(I)$
- Intuitively $A \rightarrow \alpha.B\beta$ indicates that we might see a string derivable from $B\beta$ as input
- If input $B \rightarrow \gamma$ is a production then we might see a string derivable from γ at this point

Sets of items

- C : Collection of sets of LR(0) items for grammar G'
- $C = \text{closure} (S' \rightarrow .S)$
 - repeat
 - for each set of items I in C
 - and each grammar symbol X
 - such that $\text{goto} (I, X)$ is not empty and not in C
 - ADD $\text{goto}(I, X)$ to C
 - until no more additions